

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ ПЕНЗЕНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №9

по дисциплине: "Логика и основы алгоритмизации в инженерных задачах"

на тему: "Поиск расстояний в графе"

Выполнил:

Студент группы 24ВВВ3

Мингалёв М.А.

Принял:

к.т.н., доцент, Юрова О. В.

к.т.н., Деев М.В.

Пенза 2025

Цель

Изучение обхода графа в ширину.

Лабораторное задание

Задание 1

1. Сгенерируйте (используя генератор случайных чисел) матрицу смежности для неориентированного графа G . Выведите матрицу на экран.
2. Для сгенерированного графа осуществите процедуру поиска расстояний, реализованную в соответствии с приведенным выше описанием. При реализации алгоритма в качестве очереди используйте класс **queue** из стандартной библиотеки C++.

Псевдокод

Вход: G – матрица смежности графа, v – исходная вершина.

Выход: $DIST$ – вектор расстояний до всех вершин от исходной.

Алгоритм ПОШ

- 1.1. для всех i положим $DIST[i] = -1$ пометим как "не посещенную";
- 1.2. **ВЫПОЛНЯТЬ** BFSD(v).
- 1.3 для всех i вывести $DIST[i]$ на экран;

Алгоритм BFSD(v):

- 2.1. Создать пустую очередь $Q = \{\}$;
- 2.2. Поместить v в очередь $Q.push(v)$;
- 2.3. Обновить вектор расстояний $DIST[x] = 0$;
- 2.4. **ПОКА** $Q \neq \emptyset$ очередь не пуста **ВЫПОЛНЯТЬ**
- 2.5. $v = Q.front()$ установить текущую вершину;
- 2.6. Удалить первый элемент из очереди $Q.pop()$;
- 2.7. вывести на экран v ;
- 2.8. **ДЛЯ** $i = 1$ **ДО** $size_G$ **ВЫПОЛНЯТЬ**
- 2.9. **ЕСЛИ** $G(v,i) = 1$ **И** $DIST[i] = -1$
- 2.10. **ТО**

2.11. Поместить i в очередь $Q.push(i)$;

2.12. Обновить вектор расстояний $DIST[i] = DIST[v] + 1$;

Реализация состоит из подготовительной части, в которой все вершины помечаются как не посещенные (п.1.1). В отличие от алгоритма BFS не посещенные вершины помечаем -1 , т.к. значение 0 и 1 могут быть расстояниями. Расстояние 0 – от исходной вершины до самой себя.

В самой процедуре как и в алгоритме BFS сначала создается пустая очередь (п. 2.1), в которую помещается исходная вершина, из которой начат обход (п.2.2). Расстояние до этой вершины (п.2.3) устанавливается равным 0 (расстояние до самой себя).

Далее итерационно, пока очередь не опустеет, из нее извлекается первый элемент, который становится текущей вершиной (п. 2.5, 2.6). Затем в цикле просматривается v -я строка матрицы смежности графа $G(v,i)$. Как только алгоритм встречает смежную с v не посещенную вершину (п.2.9), эта вершина помещается в очередь (п.2.11) и для нее обновляется вектор расстояния (п.2.12). Расстояние до новой i -й вершины вычисляется как расстояние до текущей v -й вершины плюс 1 (так как ребра нашего графа не взвешенные).

После просмотра строки матрицы смежности алгоритм делает следующую итерацию цикла 2.4 или заканчивает работу, если очередь пуста.

Таким образом, если вершина помещается в очередь при просмотре строки матрицы смежности на 1 -й итерации, то они находятся на 1 уровне удаленности и расстояние до этих вершин будет равным 1 .

$DIST[i] = DIST[v] + 1$, где $DIST[v] = 0$ – расстояние от исходной вершины до самой себя.

Далее, начинают просматриваться вершины первого уровня и соответствующие им строки матрицы смежности. При добавлении смежных с вершинами первого уровня вершин, расстояния до них будут равны 2 .

$DIST[i] = DIST[v] + 1$, где $DIST[v] = 1$ – расстояние от исходной вершины до вершин 1 уровня.

После того, как все вершины первого уровня будут просмотрены и извлечены из очереди, начнется просмотр вершин 2 уровня и соответствующих им строк матрицы смежности. При добавлении смежных с

вершинами второго уровня вершин, расстояния до них будут равны 3.

$\text{DIST} [i] = \text{DIST} [v] + 1$, где $\text{DIST} [v] = 2$ – расстояние от исходной вершины до вершин 2 уровня.

И так далее, алгоритм проходит вершины по уровням, пока очередь не опустеет.

Результат программы

```
E:\gitnew\laba9\laba9\Debug\laba9.exe
Input number of vertission : 10
0 1 1 0 0 1 0 0 0 0
1 0 0 1 1 1 1 1 1 1
1 0 0 0 1 0 1 0 0 1
0 1 0 0 0 0 1 0 0 1
0 1 1 0 0 1 0 1 0 1
1 1 0 0 1 0 0 1 1 1
0 1 1 1 0 0 0 0 1 1
0 1 0 0 1 1 0 0 0 1
0 1 0 0 0 1 1 0 0 1
0 1 1 1 1 1 1 1 1 0
Input start vert : 3
Path: 3
Distance from 3 to:
0 : 0
1 : 0
2 : 0
3 : 0
4 : 0
5 : 0
6 : 0
7 : 0
8 : 0
9 : 0
```

Рисунок 9 – Обход графа в ширину

Листинг

Файл 1- laba8.cpp

```
#define _CRT_SECURE_NO_WARNINGS

#include "stdafx.h"
#include <iostream>
#include <stdio.h>
#include <conio.h>
#include <cstdlib>
#include <ctime>
#include <queue>

using namespace std;

void BFSD(int** G, int numG, int* distance, int s) {
    queue<int> q;
    int v;
    distance[s] = 0;
    q.push(s);

    while (!q.empty()) {
        v = q.front();
        q.pop();
        printf("%3d", v);

        for (int i = 0; i < numG; i++) {
            if (G[v][i] == 1 && distance[i] == -1) {
                q.push(i);
                distance[i] = distance[v] + 1;
            }
        }
    }

    printf("\n\nDistance from %d to: ", s);
    for(int i = 0; i < numG; i++){
        printf("\n%3d : %3d", i, distance[i]);
    }
}

int main() {
    int** G;
    int* distance;
    int numG, current;

    printf("Input number of vertission : ");
    scanf("%d", &numG);

    distance = (int*)malloc(numG * sizeof(int));
    G = (int**)malloc(numG * sizeof(int*));
    for (int i = 0; i < numG; i++){
        G[i] = (int*)malloc(numG * sizeof(int));
    }

    for(int i = 0; i < numG; i++){
        distance[i] = 0;
        for (int j = i; j < numG; j++){
            G[i][j] = G[j][i] = (i == j ? 0 : rand() % 2);
        }
    }

    for(int i = 0; i < numG; i++){
        for (int j = 0; j < numG; j++){
```

```
                printf("%3d", G[i][j]);
            }
            printf("\n");
        }

        printf("Input start vert : ");
        scanf("%d", &current);

        printf("Path: ");

        BFSD(G, numG, distance, current);
        printf("\n\n");

        for (int i = 0; i < numG; i++)
            free(G[i]);
        free(G);
        free(distance);

        getchar();
        getchar();

        return 0;
    }
```

Вывод

В ходе выполнения лабораторной работы были разработаны программы для выполнения заданий Лабораторной работы №9. В процессе выполнения работы были использованы знания поиска расстояний в графе.

Ссылка на удаленный репозиторий GitHub:

<https://github.com/MakarMn/LiOAvIZ>